



AIngling

Development and results of an AI based phishing tool.

Samuel Rutherford

CMP309: Advanced ethical hacking

2023

Note that Information contained in this document is for educational purposes.

Abstract

Phishing is one of the most dangerous and critical threats to companies or businesses. Phishing accomplishes this through the use of impersonation and scare tactics within emails where a victim would open an email believing it to be from a trustworthy source but unknowingly clicks a malicious link or installs rouge malware onto their machine, potentially compromising their entire network. A more precise and calculated version of phishing is known as spear phishing which uses personal data from an individual to make the phishing email appear more genuine. This paper covers the development and analysis of a spear phishing tool which uses AI to generate emails based on the information enumerated from a Facebook profile.

The script was firstly developed to enumerate the victim's personal information through the use of the tool 'grep' to locate and return strings. The script was developed to use three modes catering to what level of AI interactivity the user requires. 'Manual' relied on using a template for the personal information to be passed into. 'AI' and 'special' utilised the library known as 'openai' to receive prompts relating to email generation. 'special' also performed further enumeration with the information acquired from the victim's profile. Usability aspects were also considered including command line usage and error handling.

The end product of the script (dubbed "AInglings") was overall successful but had numerous issues too. The HTML source code could not be automatically gathered from a victim's profile and had to be manually saved. Valuable information was able to be extracted from the victim's profile which greatly contributed to the phishing generation capabilities. The use of AI within the phishing generation was very effective as it was capable of finding more information and providing varied results, proving AI's deadly capability within phishing. Companies should continue to train their employees against phishing attacks and utilise numerous anti-phishing tools such as spam detection in email inboxes and phishing toolbars. Development for other features in the program for the future include improving automating information gathering and being able to harvest user's information from multiple websites.

Contents

1	Introduction	1
1.1	Background	1
1.2	Aim	2
2	Program and development	3
2.1	Overview of Procedure	3
2.2	Victim information gathering.....	3
2.2.1	Research.....	3
2.2.2	Getting the information source	3
2.2.3	Searching for information within the source code	4
2.3	Phishing email generation	5
2.3.1	Research.....	5
2.3.2	Creating manual emails.....	6
2.3.3	Utilising AI	7
2.4	Usability	9
2.4.1	Argument parsing.....	9
2.4.2	Error handling	10
3	Discussion.....	12
3.1	Overall discussion	12
3.1.1	Victim information gathering.....	12
3.1.2	Phishing email generation.....	13
3.2	Countermeasures.....	14
4	Future Work	16
5	References	17
	Appendices.....	18
	Appendix A.....	18

1 INTRODUCTION

1.1 BACKGROUND

As the digital age continues to develop, companies and organisations continue to make the mandatory switch to utilising online communications through networks. One of the most common ways of achieving this was with emails, invented by Tomlinson (1971) [1]. This invention allowed for practically instant communication between clients, colleagues and employees. Due to emails being widely used by almost all companies and businesses with a network presence, cybercriminals will often use emails as a method of launching an attack on the company. Cybercriminals will use a technique known as 'phishing' which involves the impersonation of a legitimate organisation to obtain sensitive information from an unsuspecting victim [2]. Phishing is an incredibly common cyberattack due to its ease and lacking a detriment to being performed numerous times with 3.4 billion spam emails sent every day [3]. Cybercriminals will often orchestrate an email tempting the user to click a malicious link or attachment to get the victim to install dangerous code such as ransomware onto their machine. This does mean that the best protection against phishing attempts is education and training employees into identifying phishing emails. Additionally, the use of AI generated phishing emails is on the rise due to their effectiveness at information gathering.

Often companies lack the sufficient tools and resources to provide protection against phishing email attempts. These tools should provide training to employees which include the best constructed phishing attempts. A critical aspect of improving the success rate of phishing emails is to utilise 'spear phishing' which involves using personal data found about the victim, usually online, to increase the believability and legitimacy of the fake email [4]. Identifying what methods can be used as scare tactics to make the victim more likely to urgently want to click the link are crucial. Additionally, providing variance to the phishing emails generated can be beneficial to make them appear less detectable and blend in with trustworthy emails.

Creating a tool which combines all these features would greatly improve company training against phishing attacks. The more in-depth the tool is with phishing creation; the better employees can be trained. The tool automatically gather information about the victim from a social media profile and use said information for the construction of phishing emails. To effectively create varied and legitimate emails, using AI as a method of information gathering and phishing email generation should be implemented.

The following paper will go into the research, development and reflection of the AI phishing tool development known as AInglings. The tool is developed for the purposes of training employees and researchers against phishing attempts.

The tool was developed within a kali Linux machine and consenting participants were used to have their online profiles analysed and phishing experiences documented.

1.2 AIM

This paper aims to research and develop a tool which greatly assists in spear phishing email generation. This primary aim consists of several smaller aims:

- Allow the tool to be used on a command line.
- Obtain the information about the victim in as automated way as possible.
- Specialise the tool to be used for workplace or education environments.
- Use AI to find additional information relating to the victim's personal information.
- Use AI to create phishing emails.

2 PROGRAM AND DEVELOPMENT

2.1 OVERVIEW OF PROCEDURE

To begin the development process research had taken place to find real examples of phishing taking place within a company. Pre-existing tools for gathering HTML information were used and decided if they were effective or not. Developing commands for extracting the information took place and said information was used to construct phishing emails using pre-built templates. Using the library known as openai, phishing emails were generated using a specific prompt, including the victim's personal information before allowing the AI to perform a wider variety of email generation.

2.2 VICTIM INFORMATION GATHERING

2.2.1 Research

To effectively spear phish, there must be a source of data and personal information surrounding a target. The most common method people share information publicly online would be through social media. Choosing a social media platform to build the tool's data extraction from would be crucial. Each website has numerous factors which must be considered when choosing the website:

- Popularity, users are more likely to use this website if it is popular.
- Accessible information, certain websites include much more personal information with their user's bio's such as workplace or university.

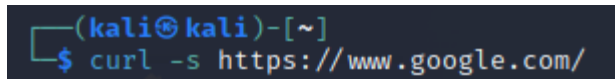
Numerous websites were considered which have their pros and cons:

- Twitter is very popular and contains some information in user's bios such as location however the information found is often limiting.
- Instagram falls under the same category as twitter however is even worse due to "posts" being centred around images, which is harder to extract information from.
- Facebook is the most popular option and allows the user to add a very large amount of data to their profile.
- LinkedIn is not as popular as the others however includes very valuable information such as email addresses, workplace, and past education.

Considering these options, the website chosen to gather information from was Facebook, mainly due to its popularity and abundance of information. Due to the tool being primarily used for targeting employees or university students, identifying workplace or university would be critical to the tool's success.

2.2.2 Getting the information source

Obtaining the information from the user's profile can be achieved in numerous ways. The first method attempted involved using the tool known as 'curl' to capture the HTTP response through the use of the command line [5].



```
(kali@kali)-[~]  
$ curl -s https://www.google.com/
```

Fig 2-1 – An example of using curl on google.com.

When attempted on a Facebook, a response was received and appeared to be the expected output however the response was lacking a vast amount of data. Due to this, an alternative method for enumeration had to be found. To further confirm that information was being left out, when manually viewing the source code for the web page, there was an abundance of information compared to using the curl method. The source code could be accessed by either right clicking on the page and navigating to the 'View page source' tab or by adding 'view-source:' in front of the URL (Fig 2-2):

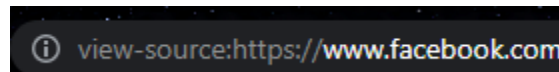


Fig 2-2 – Using view-source in the web engine Chrome.

The entire page's HTML can now be accessed allowing for more data to be acquired. A method of automatically obtaining the complete source code of page could not be found or replicated resulting in the current best method of obtaining this, is to manually save a text file of the web-page's source code. The file location would then be passed into the program as an argument so it could be accessed and analysed.

2.2.3 Searching for information within the source code

Extracting the necessary information from the source code is critical to spear phishing however it is firstly necessary to understand what information is needed first. In order to perform effective phishing, the following information is recommended or required.

- The victim's name is critical to a spear phishing attack since it allows for the email to appear personal.
- The workplace or university the victim is a part of. This acts as authority the victim thinks they must respond to.
- Location can be beneficial if it is a large company with multiple offices across the world, allowing for the phishing email to appear more realistic.

The method of accessing this data involves the use of file searching tools, such as 'grep'. Grep is a highly configurable tool which can identify and return certain strings when given very specific parameters. In order to identify where the information within the page source document is held, the surrounding strings must be identified. This is done so that the information can be found using any profile source page. Fig 2-3 is a table including the string, information and how it is found.

Information	String 1	String 2	Searched as
Name	<meta property=\"og:title\" content=	/><meta property=\"og:description\"	In between
University	Studied at	“	In between
Past education	Went to	“	In between
Workplace	"length":10,"offset":20}	.*?(?=\"}},\"__module_operation_)	In between
Location	Lives at	“	In between

Fig 2-3 – Table displaying the arguments used within the grep commands.

The grep commands used to construct the arguments can be broken down into two separate categories. Firstly, information fields such as university and location can easily be extracted due to always being before a unique string and that the information is guaranteed to be found after it. Compare this to workplace, where there is no unique string other than ‘at’, which is too common:

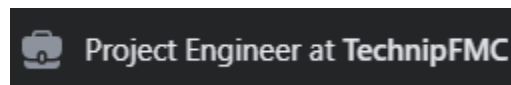


Fig 2-4 – Image showing how workplace information is stored on a Facebook profile.

Due to the work position and company both being highly variable and impossible to predict without the use of a dictionary, the HTML code surrounding the workplace information must be used within the grep statement. This process is also repeated for when acquiring the name of the victim.

2.3 PHISHING EMAIL GENERATION

2.3.1 Research

Understanding what makes effective phishing emails is essential to generating even more effective ones. For research purposes, two oil and gas engineers belonging to different companies were contacted and asked to share phishing emails they had received during work. The following image (Fig 2-4) was sent by an engineer:

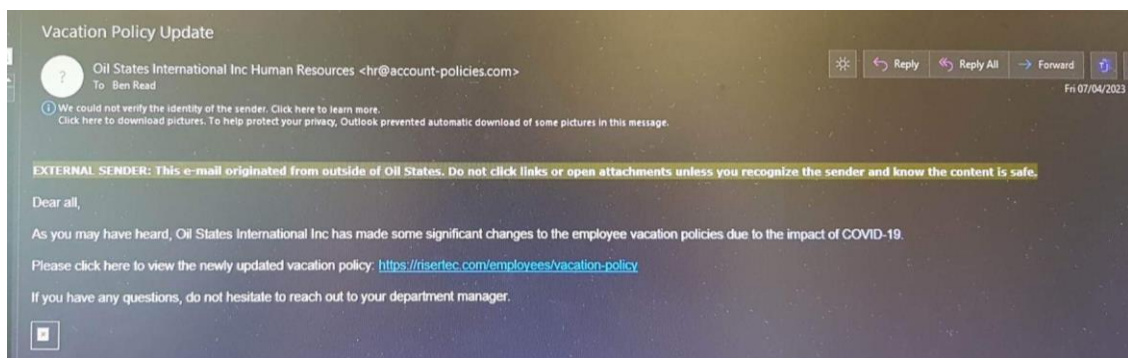


Fig 2-5 – Phishing email received by an engineer.

This phishing email attempts to impersonate the oil company, Oil and States international, specifically impersonating their human resources department. The link contained within the email directs the user to a very different website which will cause damage to the victim. The phishing email also creates a

sense of urgency with the 'Vacation Policy' being changed, which will encourage employees to click the link as they do not want to lose their precious vacation days. The next phishing email (Fig 2-5) was received by another engineer:

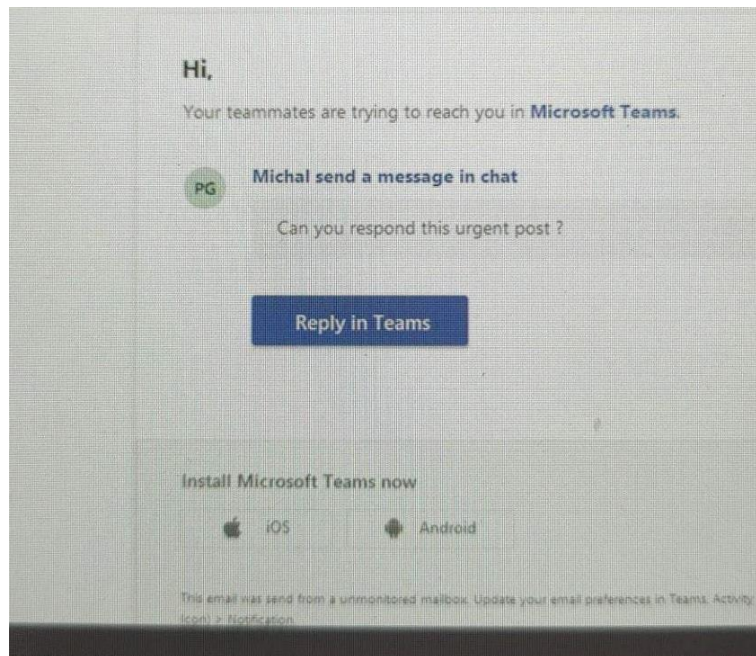


Fig 2-6 – Phishing email received by a different engineer.

Within this phishing email, Microsoft teams, a very common application used for communication between colleagues is impersonated claiming to have 'Michal' needing to urgently respond to the recipient. The 'Reply in Teams' would contain a malicious link instead of directing the recipient to Microsoft teams. Like before, the sense of urgency appears again as the made up 'Michal' is requesting you respond to his 'urgent post', making the recipient want to open this link without much hesitation. Additionally, the email is constructed to look incredibly similar to typical Microsoft teams emails, further increasing the believability of the email.

From the phishing emails provided, the key factors of what makes a convincing email can be broken down:

- Impersonation, ideally as the company. Commonly as human resources or support. Impersonating an application's automated email service also works.
- Using a link or attachment disguised as something else that the user must interact with. Critical to giving the phishing email purpose. Will be described as the payload.
- Context and storyline, help to make the phishing attempts less obvious and should encourage a user to interact with the link or attachment. Often a sense of urgency is created.
- Using graphics to make the email look professional, increasing its trustworthiness.

2.3.2 Creating manual emails

To begin the phishing generation process, the information acquired from the enumeration phase of the program is used to fill in gaps when generating the email. The program firstly checks what information is available for the program to use, by using Boolean checks:

```
# Check if work information exists
if worksetting:
```

Fig 2-7 – Checking if information about the workplace exists.

The program checks that information surrounding the workplace of the victim can be found. This check is also performed for education. Next the program begins to generate the first phishing email relating to the victim's work:

```
# Phishing email 1
mainBodyWrite('WorkPhish1.txt', "URGENT - MISSING DOCUMENTS - " + work + "\n",
```

Fig 2-8 – Phishing email being constructed.

A text file is created where the email will be saved and what will be used as the subject which includes the victim's workplace and 'URGENT' is included to scare and increase the likelihood that the user would open the email. Next the main body of the email is constructed with the following strings:

```
"Good morning " + (name.decode('utf-8')) + ", Recently numerous documents have gone missing from " +
work + " systems. Please find the attached PDF if you are believed to be responsible for the loss of any
documents. Regards, Human resources"
```

The victim's name is used to increase the legitimacy of the phishing attempt as personalising the email can make it appear more trustworthy. The urgency of the email is further exaggerated by providing a storyline and context to the email by potentially putting blame upon the victim, and that they can prove their innocence by clicking on the attachment. The file containing the generated email can be seen within the respective output text files located in the same directory as the script.

Another work-related phishing email is generated, using a different storyline and context however instead of carrying a payload through an attachment, this time it is achieved using a link. The program also checks if the victim has university information available and generates two emails surrounding the university and victim (Appendix A).

2.3.3 Utilising AI

With AI being recently used to automate tasks within industries, its capabilities for being used to generate phishing emails should be greatly used. The AI library being used is known as openai [6] and communicates using its API. A prompt must be sent to the AI in order to receive a response. For the workplace phishing email, within this prompt, the AI is given the context and story similar to the previously made phishing emails:

```
"Can you make an email which is about accusing the recipient (known as " + (name.decode('utf-8')) + ") of
plagiarism at " + uni + "? Please add that the information surrounding their accused plagiarism is included
within an attached PDF and they should open it ASAP. Please also add that this email is from 'Student
Support' and it is an automated message, i.e. will not respond and do not mention anything about
contacting themselves."
```

Utilising AI should result in the script being able to meticulously construct a convincing email due to its abundance of professionally made emails online it can access. When generating the response, it can be configured to limit its random variance when generating answers using the 'temperature' parameter:

```
response = openai.Completion.create(
    engine="text-davinci-003",
    prompt=prompt,
    max_tokens=100,
    n=1,
    stop=None,
    temperature=0.1,
)
```

Fig 2-9 – Generating the AI response.

The temperature can be controlled by a number between 0.0 and 1.0, with 1.0 resulting in the most variance and 0.0 making answers stay very similar. 0.0 was chosen for this mode due to wanting consistent emails to be generated. The result can be seen in Fig 2-8:

```
Subject: Plagiarism Allegation at Abertay University

Dear Sam Rutherford,

This is an automated message from Student Support at Abertay University.

We are writing to inform you that we have received an allegation of plagiarism against you.

The information surrounding this allegation is included in the attached PDF. We advise that you open it as soon as possible.

Please note that this email is automated and we will not respond to any replies.
```

Fig 2-10 – AI generated phishing email relating impersonating the victim's university.

An aspect from the manually generated phishing emails which was missing was the use of graphics from the source which the email is impersonating. The AI can be used to find resources that relate to the university or workplace that the victim belongs too. The following prompt is used to find the logo for the organisation, which in this case is the victim's university:

```
# Get logo for uni to appear more realistic
imagePrompt = "Can you send me a link which contains an image that has the logo| for " + uni+ "?"
imageResponse = openai.Completion.create(
    engine="text-davinci-003",
    prompt=imagePrompt,
    max_tokens=100,
    n=1,
    stop=None,
    temperature=0.0,
)
# Extract the generated answer from the response
imageAnswer = imageResponse.choices[0].text.strip()
```

Fig 2-11 – Using AI to find the logo of the victim's university.

The link is then saved to the document alongside the rest of the AI generated email. The success rate of the link returning the correct image is not 100%.

The final mode to be implemented was dubbed 'special' which allowed for the AI to have much more freedom in the emails it generates. This is achieved by giving the AI a much broader prompt:

"Can you make me a common email that a university's management would send to their students, which involves needing to view an attachment or click a link. Additional information to include in this email:

Name of the recipient is " + (name.decode('utf-8')) + ", the university is called "+uni+ " and sign the email with student support. If you can find can any additional information about the company to improve the quality of the email, please do."

This iteration of AI generation allows for a greater amount of variation in the results. This prevents the same context from appearing again and makes each email seem more professional and genuine.

```
Dear Sam Rutherford,  
  
We hope you are doing well and enjoying your studies at Abertay University.  
  
We would like to take this time to remind you of an important announcement from Abertay University's management.  
  
In order to view the information, please click on the link provided or download the attachment.  
  
At Abertay, we strive to ensure that you have the best learning experience possible.  
  
If you have any questions or concerns, please don't hesitate to contact Student Support at studentsupport@abertay.ac.uk.  
  
Thank you for your cooperation and we wish you all the best in your studies.  
  
Sincerely,  
Abertay University Student Support
```

Fig 2-12 – Example output from using the special mode.

Within this example, the AI also utilises its ability to find information online to further improve its legitimacy when it mentions the student support email address, including the correct domain name too.

2.4 USABILITY

2.4.1 Argument parsing

The script makes use of the library known as 'argparse' which is used to allow specific values to be parsed into the script through the command line. This also allows to keep user input to a minimum. An example of how to run the script is seen in Fig 2-12:

```
(kali㉿kali)-[~/Desktop/ScriptingProject]  
$ python AIngling.py special samSource.txt
```

Fig 2-13 – Example of parsing arguments into the script.

Name	Help	Required	Example
Mode	Choose whether to use AI, manual, or special generated phishing	Yes	Special

Source	Name of the target source code file	Yes	samSource.txt
info	Display help information.	No	--info
Verbose	Display more information as the script operates.	No	-v

Fig 2-14 – Table of arguments which can be parsed into the script.

2.4.2 Error handling

Error handling is present within the program to alert the user on how errors arise and how they can be fixed. Errors will most commonly occur from two scenarios:

- The user incorrectly uses the script, such as incorrectly inputting arguments. This is handled using else statements to identify when the expected parameters have not been implemented.

```

25 else:
26     # error handling, inform the user to use 'ai', 'manual' or 'special'
27     print("Please use 'ai', 'manual' or 'special' for the mode.")

```

Fig 2-15 – Else statement being used to hand the error when none of the specified modes are chosen.

- Information within the source code text document is lacking. When performing the grep searches, if no string can be returned, the script informs the user.

```

# University
command = "cat " + source + " | grep -oP '\"Studied at\\K.*?(?=\\\"))'"
try:
    output = subprocess.check_output(command, shell=True, text=True)
    uni = output.strip()
    vprint(args, "University:", uni)
    edusetting = True
    # if have time, find course
except subprocess.CalledProcessError:
    print("Failed to retrieve university information.")

```

Fig 2-16 – Error handling utilising the subprocess library to spot the error.

2.4.3 Information display

The script utilises the argument known as ‘verbose’ which allows the user to control the amount of information which is displayed to them within the terminal. Every time a statement which will only print if verbose is selected, the function known as ‘vprint’ checks if the verbose argument is enabled:

```
14 # prints message when verbose is activated
15 def vprint(args, *message):
16     if args.verbose:
17         print(*message)
```

Fig 2-17 – Function responsible for handling if the value should be printed.

If the verbose argument had been passed into the script, the message would print.

3 DISCUSSION

3.1 OVERALL DISCUSSION

The creation and research of AInglings was reasonably successful. The tool was fulfilled its goals of spear phishing a victim using an online profile and generating a multitude of different phishing emails and resources which a cybercriminal could use to make convincing emails.

3.1.1 Victim information gathering

The victim information gathering aspect of the script was responsible for numerous failures of the project. The research however was successful as it was insightful into the best methods and websites phishers can use when attempting to gather a user's information. Facebook was the best option due to vast popularity among all demographics however LinkedIn should have been considered more due to the nature of the application being surrounded around employment and current or past educations, which this script focuses on.

Arguably the biggest failure of the project was the lack of an easy way to acquire the source code of the page, however the issues are deep and often lead to more issues. Being easy to use and relying on minimal user input was one of the project objectives however the program fails at this due to not being able to use curl since the output from the tool is greatly reduced. The reasoning behind this is most likely due to Facebook detecting that the HTTP response is not going towards a browser and therefore reduces the output to protect user's data. Using the view-source: method is an acceptable alternative however there are issues which must be considered too. Cookies and messenger data will also be found when using view-source:URL which can result in mass amounts of unwanted data being harvested. Additionally, the profile information on the Facebook page can change assuming the user is logged in. Below compares the information found on a user's profile when logged in compared to being logged out:

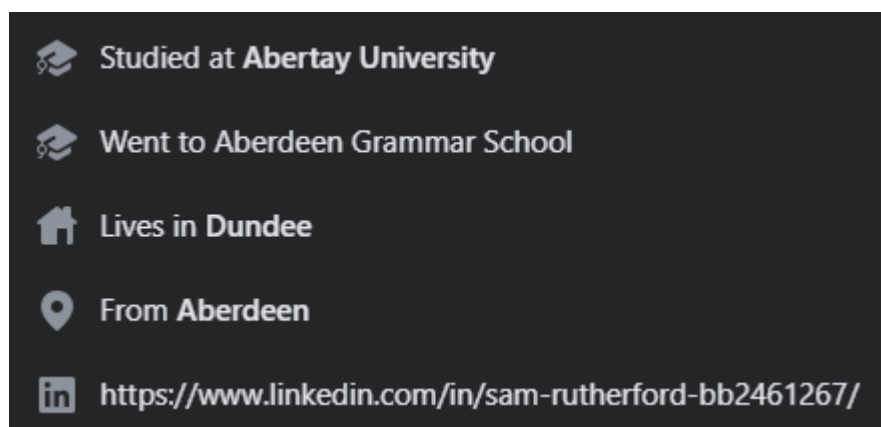


Fig 3-1 – User's profile information viewed when logged in.

About

Work



No workplaces to show

University



Studied at **Abertay University**
Ethical Hacking · School year 2024

High School



Went to **Aberdeen Grammar School**

Fig 3-2 - User's profile information viewed when logged out.

The difference in information can result in inconsistent outcomes between profiles. Due to this, the development and testing of the script were performed using a logged-out browser. Facebook's reasoning for this difference in personal information being viewable is most likely to combat users' personal information being harvested.

Utilising grep was for finding the victim's information was effective however could be improved. The information found did greatly help to construct accurate phishing emails however more personal information could have been acquired such as degree and other websites. Accessing this information would have been challenging due to the information available within the source code varying on the user's logged in state. This personal information would have been able to improve the AI's ability to create even more convincing phishing emails.

3.1.2 Phishing email generation

The phishing email generation was successful, and the research greatly contributed to its success. From the research, a methodology to creating effective phishing emails could be made:

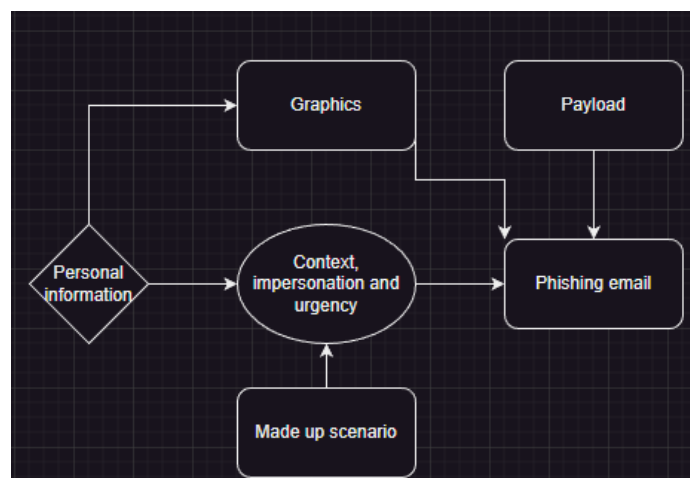


Fig 3-3 – Flow chart diagram of how phishing emails can be constructed.

Utilising this methodology was key to creating effective phishing emails, the first of which were manually constructed. These were reasonably effective however were extremely repetitive due to relying on a template. Additionally, if there were any formatting issues from performing the grep commands, this was reflected in the output. This was not the case when utilising AI to assist with the email generation as the AI was capable of having a varied output of generated emails and was capable of automatically fixing formatting issues. The AI was also used to locate logos of the victim's organisation which was used to further the believability and improve the professionalism of the emails. Finally, the last mode which the program could use allowed the AI to generate a wider variety of emails with a broader prompt. This also allowed the AI to further the enumeration by using the information acquired in order to improve the fake legitimacy of the emails. This was incredibly effective as in an example the script was able to identify the student support email address for the university of the victim.

3.1.3 Usability

Sufficient considerations for the usability of the script had successfully been implemented allowing for improved user experience. The argument parsing and configurable information display allow for the user to control their experience from the command line while the error handling allows the user to fix errors of their own creation by misusing the script.

The script runs efficiently due to using functions to avoid repeating code. Overall, the script's run time is fast however varies depending on the mode. In order to determine what was causing an increase in time, 7 tests were ran timing for the script to complete for each of the modes within the script before calculating an average:

Mode	Time (s)
Manual	0.53
AI	19.77
Special	20.04

Fig 3-4 – Time taken for the script to complete for each mode.

From the results, it can be clear that manual was by far the quickest mode to run. This is most likely due to its lack of needing to use the AI. This does result in the performance of the script being severely bottlenecked due to waiting on a response from the AI API.

3.2 COUNTERMEASURES

Protecting against phishing is vital and can be broken down into separate categories. Firstly, protecting against specifically spear phishing attacks often comes down to the individual or the social media platform where the information is most likely acquired. Having a small digital footprint reduces the risk of information being leaked to be used in phishing emails and certain social media websites will make it harder for automated methods of obtaining information.

Protecting against impersonation attempts for phishing comes down to the individual and the company they are employed too. Numerous countermeasures can be applied such as utilising spam filters to

detect suspicious emails but most importantly, employees should undergo training to identify and avoid phishing emails [7]. While AI can be used to create phishing emails, it can also be used to identify and prevent them through the use of detecting patterns of words and phrases and machine learning [8]. Certain anti-phishing tools exist such as an anti-phishing toolbar which can be used in the user's browser to alert the user if they are accessing a known phishing website [9].

Identifying if a link is suspicious within an email is suspicious can be achieved by hovering over the link within the email [10]. This can be seen within the example email sent by an engineer:

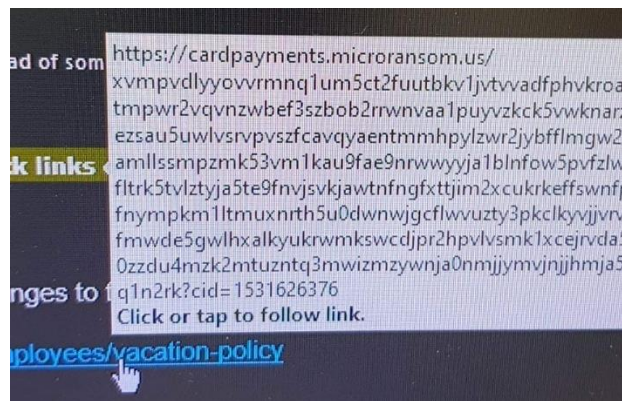


Fig 3-5 – Link within phishing email showing the real address.

The link appears to take the user to a website where their card details would potentially be stolen.

Maintaining strong password security within company accounts prevents the account from becoming compromised and being used for even more dangerous phishing methods as the attacker can now send emails from a trustworthy address. Two-Factor authentication is another method of preventing unwanted access to company accounts and should absolutely be used [9]. In the event a user does open a payload with malware on it, antivirus software should be used to mitigate the damage done.

4 FUTURE WORK

Numerous aspects of the program could be improved greatly. To begin the information gathering process could be greatly streamlined and expanded upon:

- More information from the user's profile could be extracted including colleague names and university degree. This would greatly expand upon improving the professionalism which can be achieved within the emails and make them appear more trustworthy.
- Detecting when links to other profiles across different websites exist and performing scans on those as well. This would allow for even more enumeration to take place such as identifying work emails and other colleagues. If the link went to another website other than Facebook, more grep statements would have to be used due to websites using different methods of holding the user's information in HTML. Additionally, links to the victim's organisation could be passed into the AI to allow it to perform further enumeration.
- Detecting and correctly handling when a the submitted source code of a profile contains the user's cookies. This should aim to remove the cookies data from the source text file to avoid unwanted data being extracted.

Certain aspects of the phishing generation could also be improved:

- Improving the generation of graphics and using the AI to identify and use the format that the email is impersonating. The format which the phisher's used made incredibly effective and trustworthy emails which would make more victim's click the link or attachment.
- Integrating the payload being used into the text document more effectively. This would improve the user's experience by making the tool more automated.

5 REFERENCES

- [1] Jennings, V. (2016, March 10). *A Brief History of Email: Dedicated to Ray Tomlinson*. Retrieved May 21, 2023, from phrasee: <https://phrasee.co/news/a-brief-history-of-email/#:~:text=On%20October%2029th%201969%2C%20the,computer%20to%20computer%20on%20ARPANET.&text=It%20was%201971%20when%20Ray,creating%20ARPANET's%20networked%20email%20system.>
- [2] Phishing.org. (2023, May 21). *What is phishing?* Retrieved from Phishing.org: <https://www.phishing.org/what-is-phishing>
- [3] Griffiths, C. (2023, May 5). *The Latest 2023 Phishing Statistics (updated May 2023)*. Retrieved May 21, 2023, from aag: <https://aag-it.com/the-latest-phishing-statistics/>
- [4] Irwin, L. (2023, January 31). *The 5 Most Common Types of Phishing Attack*. Retrieved May 21, 2023, from it governance: <https://www.itgovernance.eu/blog/en/the-5-most-common-types-of-phishing-attack>
- [5] Aleksic, M. (2021, September 16). *Linux curl Command Explained with Examples*. Retrieved May 21, 2023, from phoenixNAP: [https://phoenixnap.com/kb/curl-command#:~:text=curl%20\(short%20for%20%22Client%20URL,client%2Dside%20URL%20transfer%20library](https://phoenixnap.com/kb/curl-command#:~:text=curl%20(short%20for%20%22Client%20URL,client%2Dside%20URL%20transfer%20library)
- [6] Openai. (n.d.). *API reference*. Retrieved May 18, 2023, from Openai: <https://platform.openai.com/docs/api-reference?lang=python>
- [7] Groot, J. D. (2023, May 5). *4 Steps to Prevent Phishing Attacks (According to 33 Experts)*. Retrieved May 22, 2023, from FORTRA: <https://www.digitalguardian.com/blog/phishing-attack-prevention-how-identify-prevent-phishing-attacks#:~:text=Install%20an%20antivirus%20solution%2C%20schedule,Encrypt%20all%20sensitive%20company%20information.>
- [8] M, S. (2023, April 1). *"AI-Powered Defense: How Artificial Intelligence Can Prevent Phishing Attacks"*. Retrieved May 22, 2023, from LinkedIn: <https://www.linkedin.com/pulse/ai-powered-defense-how-artificial-intelligence-can-prevent-suhail-m/>
- [9] Sutarwala, U. (2022, October 19). *Best Countermeasures Against Phishing Attacks in Today's High-Risk Business Landscape*. Retrieved May 22, 2023, from ITSecurity Wire: <https://itsecuritywire.com/future-ready-articles/best-countermeasures-against-phishing-attacks-in-todays-high-risk-business-landscape/>
- [10] Lindley, P. (2015, September 2015). *Spear Phishing Attacks and Countermeasures*. Retrieved May 22, 2023, from INFOSEC: <https://resources.infosecinstitute.com/topic/spear-phishing-attacks-and-countermeasures-to-mitigate-against-them/>

APPENDICES

APPENDIX A

```
1 URGENT - PLAGIARISM - Abertay University|
2 Abertay University
3 Good morning Sam Rutherford
4 , Recently, coursework you submitted is currently being reviewed for plagiarism.
5 Please see the attached PDF for information on how you can rebuttal this and what work is being reviewed. Regards, Student support
6 Insert payload here
```

Fig 1-1 – UniPhish1.txt

```
1 URGENT - NO RESPONSE - Abertay University
2 Abertay University
3 Good morning Sam Rutherford
4 , I have received numerous complaints about your failure to respond to your messages on Microsoft Teams.
5 This is not acceptable, and serious action will be taken unless you respond ASAP at [MALICIOUS LINK]. Regards, Student support
6 Insert payload here
7 |
```

Fig 1-2 UniPhish2.txt